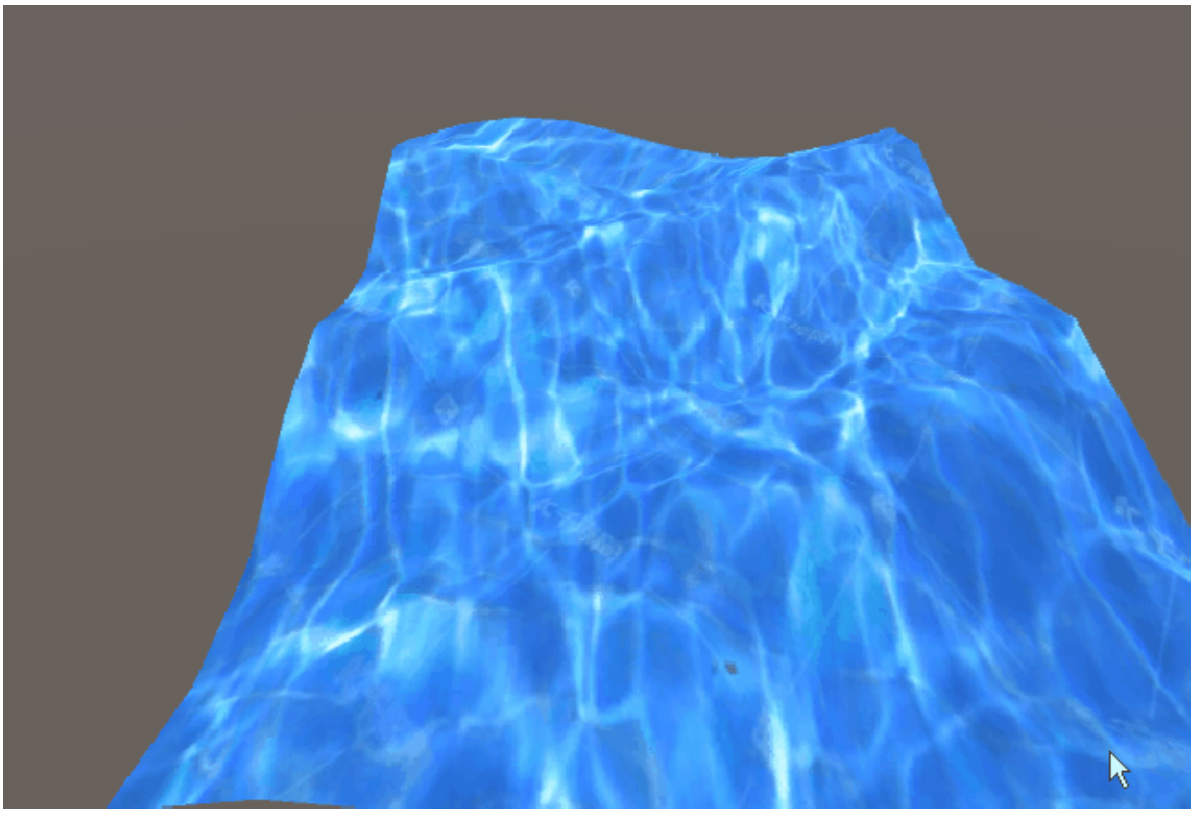
流体模拟实验报告

19220205金桢桢

正弦波模拟

利用mesh类的顶点变化模拟水平面



特点

圆润、平滑

其实只有mesh顶点的y坐标变化，x/z并没有移动，所以效果并不真实，像是理想中风平浪静的小池

原理

$$h(x,z,t) = -y_0 + \sum_{i=1}^{N_w} A_i cos(k_{i_x} x + k_{i_z} z - w_i t)$$

- 其中 N_w 是波的总数
- A_i 是第i个波的振幅
- $\overrightarrow{k_i} = (k_{i_x}, k_{i_z})$ 是波矢量
- w_i 是其脉冲值（pulsation）
- y_0 是自由表面的高度

代码实现

类变量

建立一个100*100的mesh

```
private MeshFilter meshFilter;
private MeshRenderer render;
private Mesh mesh;
Vector3[] newVertices = new Vector3[100];
Vector3[] SLVertices = new Vector3[100];
int[] newTriangles = new int[486];
Vector2[] newUV = new Vector2[100];
Vector3[] newNormals = new Vector3[100];
```

Start()函数

```
void Start()
    {
        meshFilter = GetComponent<MeshFilter>();
        mesh = meshFilter.mesh;
        mesh.Clear();
        mesh.vertices = CreateVertices(10);
        mesh.triangles = CreateAnlges();
        mesh.uv = CreatUVS();
        mesh.normals = CreateNormals();
    }
```

设定顶点位置

```
private Vector3[] CreateVertices(int arr_j)
    {
        float z = -.5f;
        for(int i = 0; i < 10; i++)
        {
            z += .5f;
            for (int j = 0; j < 10; j++)
            {
                newVertices[arr_j * i + j].x = j;
                newVertices[arr_j * i + j].y = 0;
                newVertices[arr_j * i + j].z = z;
            }
        }

        return newVertices;
    }
```

```
private int[] CreateAnlges()
    {
        int e = 0;
        for(int i = 0; i < 89; i++)
        {
            if (i % 10 != 9)
            {
                newTriangles[e] = i;
                newTriangles[e + 1] = i + 10;
```

```
                newTriangles[e + 2] = i + 11;
                newTriangles[e + 3] = i + 11;
                newTriangles[e + 4] = i + 1;
                newTriangles[e + 5] = i;
                e += 6;
            }
        }
        return newTriangles;
    }
```

给mesh贴图

```
 private Vector2[] CreatUVS()
    {
        float u = 0;
        float v = 0;
        for(int i = 0; i < 100; i++)
        {
            newUV[i].y = v;
            if (i % 10 != 0)
            {
                u += 0.1f;
                newUV[i].x = u;
            }
            else
            {
                u = 0;
                newUV[i].x = u;
                v += .1f;
            }
        }
        return newUV;
    }
```

设定法线

```
private Vector3[] CreateNormals()
    {
        for (int i = 0; i < 100; i++)
            newNormals[i] = Vector3.up;
        return newNormals;
    }
```

Update()函数

以最简单的方式实现正弦波波动，参数固定

```
void Update()
    {
        //wave
        for (int i = 0; i < 100; i++)
            newVertices[i].y = .2f * Mathf.Sin(1.5f * newVertices[i].x + 1.5f *
newVertices[i].z + Time.time);
        mesh.vertices = newVertices;
    }
```

参考资料：

> 真实感水体渲染技术总结 - 知乎 (zhihu.com)

> Chapter 1. Effective Water Simulation from Physical Models | NVIDIA Developer